



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA

VICERRECTORADO ACADÉMICO

COORDINACIÓN GENERAL DE PREGRADO

INGENIERÍA INFORMÁTICA

ASIGNATURA: INGENIERÍA DEL SOFTWARE II

SECCIÓN 1

“Fundamentos de la Gestión de Proyectos y Productos de Software”

DOCENTE:

Félix Márquez

ALUMNOS:

Luis Millán

V - 30.040.213

Endrys Flores

V - 30.451.556

Robert Castro

V - 30.994.049

Jesus Báez

V - 26.753.871

Ciudad Guayana, Septiembre del 2026.

ÍNDICE.

INTRODUCCIÓN	3
Ejercicio 1	4
Ejercicio 2	7
Ejercicio 3	11
Ejercicio 4	15
CONCLUSIONES	18

INTRODUCCIÓN

La creación de sistemas de software ha evolucionado de ser una actividad meramente técnica a transformarse en un proceso de gestión estratégica centrado en solucionar problemas concretos y en la provisión constante de valor. Históricamente, la denominada "Crisis del Software" mostró que la causa principal del fracaso en los proyectos no era la falta de calidad en la codificación o compilación, sino en una gestión deficiente de requisitos, la desconexión con las necesidades reales de los usuarios y la inflexibilidad metodológica.

Este informe examina de manera analítica y reflexiva las bases de la gestión de proyectos y productos de software mediante cuatro casos de estudio aplicados. Se analiza la obtención y conversión de requerimientos conforme a normas internacionales como ISO/IEC/IEEE 29148 y SWEBOK V4, el desequilibrio sistémico en las cuatro dimensiones del software (Personas, Proceso, Producto y Tecnología), la evolución del modelo prescriptivo del PMBOK 6ª Edición hacia el enfoque adaptativo basado en principios del PMBOK 7ª Edición, y la reducción del riesgo contemporáneo de desarrollar sistemas técnicamente adecuados que son finalmente rechazados por los usuarios.

Ejercicio 1

Traducción de Necesidades a Requerimientos y Actores

Contexto: Un hospital público necesita reducir el tiempo de espera de los pacientes en urgencias.

Instrucciones: Identifique al menos 4 actores (stakeholders) clave en este escenario. Para cada actor, redacte una "Necesidad" (el problema encontrado) y tradúzcala a un Requerimiento específico del software (funcional o no funcional).

Reflexión: ¿Cómo podría la IA o el prototipado ayudar a validar estos requisitos antes de programar?

Respuesta: Se reconocen 4 actores fundamentales, abarcando la perspectiva de los usuarios finales, el personal administrativo, el personal operativo y aquellos en oposición o que resisten el cambio.

Actor 1: Paciente de Urgencias (Usuario Final / Directo)

- **Tipo de Actor:** Interesado clave / Beneficiario directo.
- **Necesidad (Problema encontrado):** Ignora cuánto tiempo tardará en ser atendido y desconoce su lugar en la fila de espera, lo que genera ansiedad, aglomeración en la sala y reclamos constantes al personal.
- **Requerimiento Específico:**
 1. **Tipo:** Funcional.
 2. **Especificación (ISO 29148):** El sistema deberá enviar notificaciones en tiempo real vía SMS o WhatsApp al teléfono móvil del paciente, indicando su posición en la fila de espera y una estimación dinámicamente calculada del tiempo de atención.

Actor 2: Médico Triage (Usuario Operativo)

- **Tipo de Actor:** Usuario primario del sistema.
- **Necesidad (Problema encontrado):** Pierde demasiado tiempo registrando manualmente los síntomas del paciente e interpretando la gravedad de los casos, lo que ralentiza la clasificación inicial y congestiona la entrada a urgencias.
- **Requerimiento Específico:**
 1. **Tipo:** Funcional.
 2. **Especificación (ISO 29148):** El sistema deberá proveer un módulo de clasificación de triage digital en el que el médico seleccione los signos vitales y síntomas mediante opciones predeterminadas, asignando automáticamente un nivel de prioridad de atención según la escala Manchester.

Actor 3: Director General / Gerente del Hospital (Patrocinador / Decisor)

- **Tipo de Actor:** Stakeholder estratégico.
- **Necesidad (Problema encontrado):** No cuenta con visibilidad en tiempo real de los cuellos de botella en el flujo de atención, lo que le impide tomar decisiones informadas sobre la reasignación de personal en turnos de alta demanda.
- **Requerimiento Específico:**
 1. **Tipo:** No Funcional (Atributo de Calidad / Usabilidad y Rendimiento).
 2. **Especificación (ISO 29148):** El sistema deberá generar e imponer un panel de control (*dashboard*) analítico que actualice cada 30 segundos las métricas de tiempo promedio de espera, tasa de ocupación de camas y volumen de pacientes atendidos, con un tiempo de respuesta de carga inferior a 2 segundos.

Actor 4: Sindicato de Enfermería / Personal Resistente al Cambio (Actor Restrictivo)

- **Tipo de Actor:** Stakeholder en oposición indirecta / Resistente al cambio.
- **Necesidad (Problema encontrado):** Temor a que el nuevo software aumente su carga de trabajo administrativo, requiere capacitación compleja o sea utilizado como una herramienta para fiscalizar o sancionar su rendimiento laboral.
- **Requerimiento Específico:**
 1. **Tipo:** No Funcional (Usabilidad y Accesibilidad).
 2. **Especificación (ISO 29148):** El diseño de la interfaz para el registro de pacientes deberá requerir como máximo 3 clics por pantalla y permitir la completación del formulario en un tiempo promedio no mayor a 45 segundos, sin solicitar campos redundantes ya existentes en la base de datos.

Reflexión: De acuerdo con el marco del SWEBOK V4 y la Curva del Costo del Cambio de Boehm, identificar un error o malentendido en la fase de especificación puede costar hasta 100 veces menos que solucionarlo en producción. La combinación de prototipado ágil y herramientas de Inteligencia Artificial (IA) mejora esta validación antes de la etapa de construcción:

1. Aporte del Prototipado Rápido.

- **Reducción de la Ambigüedad:** Emplear herramientas de prototipado interactivo (como Figma o Balsamiq) facilita mostrar al médico de triage o al equipo de enfermería una simulación interactiva del sistema.
- **Validación de Experiencia de Usuario (UX) e Intuición:** Permite verificar si la exigencia de registrarse en "45 segundos" es realmente factible en la práctica. El usuario

se conecta con la maqueta y proporciona comentarios sobre la colocación de botones y claridad antes de escribir cualquier línea de código.

2. **Aporte de la IA Generativa en la Elicitación y Validación**

- **Detección de Inconsistencias y Vacíos Legal/Operativos:** Al introducir las especificaciones escritas de los requisitos en un modelo de lenguaje (LLM), este puede evaluar la consistencia entre ellos e identificar situaciones límites no consideradas (p. ej. "¿Qué pasa si el paciente no tiene un teléfono inteligente para recibir avisos?")
- **Generación Automática de Historias de Usuario y Prototipos:** La IA facilita la conversión de descripciones en texto sin formato en prototipos iniciales o código de muestra, agilizando el ciclo de retroalimentación con los responsables del hospital y adaptando la dirección del producto de forma rápida.

Ejercicio 2

Análisis de las 4 Dimensiones y Errores Clásicos

Contexto del Caso: Una startup decide desarrollar una app de delivery. El gerente técnico impone el uso de una arquitectura de microservicios en Kubernetes (Tecnología), pero el equipo solo sabe programar en PHP monolítico (Personas). No definieron un proceso de gestión de cambios (Proceso) y el cliente pide agregar una función de "pagos con criptomonedas" a mitad del desarrollo sin ajustar el presupuesto (Producto/Alcance).

1. Desbalance en las 4 dimensiones de Pressman

Pressman plantea que Personas, Proceso, Producto y Tecnología deben permanecer alineadas; el caso muestra un desbalance simultáneo en las cuatro, lo cual explica por qué el fracaso del proyecto no depende de una sola causa sino de un efecto sistémico.

Dimensión	Estado en el caso	Desbalance identificado	Efecto proyectado
Personas	El equipo domina PHP monolítico; no tiene experiencia en microservicios ni en Kubernetes.	Sobre-tecnificación impuesta desde arriba sin evaluar la capacidad real del capital humano.	Curva de aprendizaje larga, frustración, alta probabilidad de errores de arquitectura y retrasos.
Tecnología	Se exigen microservicios + Kubernetes, un stack de alta complejidad operativa.	La tecnología se eligió por moda/prestigio técnico, no en función de las personas ni del proceso disponibles.	Sobre-ingeniería (over-engineering) para el tamaño de una startup; costo de infraestructura y DevOps innecesario.
Proceso	No existe un proceso de gestión de cambios (change management) definido.	Vacío total de marco de trabajo: no hay Scrum, Kanban ni comité de cambios que regule nuevas peticiones.	Cualquier petición del cliente entra sin control, generando caos de alcance y pérdida de trazabilidad.
Producto / Alcance	El cliente añade "pagos con criptomonedas" a mitad del desarrollo sin ajustar presupuesto.	El producto crece sin que el triángulo de hierro (alcance-costo-tiempo) se renegocie.	Scope creep clásico: se compromete la fecha de entrega, el presupuesto o la calidad del producto final.

El desbalance no está aislado en una dimensión: la Tecnología se sobredimensiona sin considerar a las Personas; la ausencia de Proceso deja sin filtro los cambios de Producto/Alcance. Es exactamente el pensamiento sistémico que exige el PMBOK 7: un cambio en una dimensión repercute en todas las demás.

2. Errores clásicos identificados

Se identificaron estos errores

Error clásico (Brooks / CHAOS Report)	Evidencia en el caso	Por qué aplica
1. Optimismo injustificado	El gerente técnico impone Kubernetes/microservicios sin considerar que el equipo solo sabe PHP monolítico.	Se subestima la complejidad real de adoptar una arquitectura distribuida sin la curva de aprendizaje ni el tiempo que exige (Ley de Hofstadter).
2. Scope Creep (alcance no controlado)	El cliente pide "pagos con criptomonedas" a mitad de camino sin renegociar presupuesto ni cronograma.	Es la definición textual del error: adición continua de requisitos sin ajustar tiempo/costo.
3. Ausencia de proceso / falta de gobierno del cambio	No existe un proceso definido de gestión de cambios que filtre o evalúe nuevas solicitudes.	Sin un canal formal, cualquier petición se cuela en el alcance; es la puerta de entrada que habilita el propio scope creep.
4. Falta de participación real del usuario / desalineación técnica	La decisión de arquitectura la toma unilateralmente el gerente técnico, sin validar con el equipo que la ejecuta ni con las necesidades reales del negocio.	Se construye sobre lo que el gerente técnico "cree conveniente" (prestigio tecnológico) y no sobre la capacidad real de entrega ni la necesidad de negocio.

3. Solución de mitigación inmediata (rol de Líder Servicial)

Como Gerente de Proyectos moderno la intervención debe ser inmediata y accionable en las cuatro dimensiones:

a) Sobre Tecnología y Personas — detener y evaluar (Spike técnico).

- Congelar la decisión de Kubernetes/microservicios hasta ejecutar un spike de 1–2 semanas que mida el costo real de capacitar al equipo vs. adoptar un stack intermedio (p. ej. PHP

modular con contenedores simples, o un monolito bien desacoplado) coherente con la capacidad actual.

- Negociar con el gerente técnico: la arquitectura se adapta al equipo y al momento del negocio (principio de Adaptación/Tailoring del PMBOK 7), no al revés.

b) Sobre Proceso — instaurar un Comité de Control de Cambios (CCB) ligero.

- Crear de inmediato un flujo simple de gestión de cambios: toda solicitud nueva (como "pagos con criptomonedas") se registra, se evalúa el impacto de tiempo/costo/riesgo y se prioriza en el backlog, nunca se ejecuta "por fuera".
- Adoptar un marco ágil ligero (Kanban o Scrum con sprints cortos) que dé visibilidad continua del avance real y permita decir 'sí, pero...' en vez de un 'sí' o 'no' ciego.

c) Sobre Producto/Alcance — renegociar el Triángulo de Hierro.

- Presentar al cliente el impacto cuantificado del cambio (tiempo, costo, riesgo) y negociar: mover la función a una fase 2, ampliar presupuesto/plazo, o sacrificar otra funcionalidad de menor valor (trade-off explícito, no silencioso).
- Formalizar el acuerdo en un documento breve de cambio de alcance firmado por cliente y equipo, evitando que el compromiso quede solo verbal.

d) Rol transversal del Líder Servicial.

- Facilitar, no imponer: mediar entre el gerente técnico y el equipo para que la decisión de arquitectura sea consensuada, no impuesta desde el poder jerárquico.

- Proteger al equipo de la sobrecarga y la ambigüedad, garantizando seguridad psicológica para que expresen que no dominan Kubernetes sin temor a represalias.
- Mantener foco en el valor (principio Value del PMBOK 7): priorizar lo que hace viable el negocio (lanzar el MVP de delivery) sobre lo que luce impresionante en el CV técnico (microservicios) o lo que el cliente pide de forma reactiva (cripto).

Como Reflexión: El caso es un ejemplo de libro de cómo el desbalance simultáneo en las 4 dimensiones potencia los errores clásicos entre sí. La mitigación no es técnica sino de liderazgo: bajar la ambición tecnológica al nivel del equipo, instalar un proceso mínimo de control de cambios y renegociar abiertamente el alcance con el cliente, en vez de absorber el cambio en silencio.

Ejercicio 3

Análisis Comparativo del Enfoque de Gestión de Requisitos (PMBOK 6 vs. PMBOK 7)

Introducción al Análisis.

La evolución de la Guía del PMBOK del Project Management Institute (PMI) representa una reorientación epistemológica fundamental en la dirección de proyectos. Este ejercicio analiza cómo el cambio de un paradigma basado en **procesos prescriptivos (PMBOK 6)** a uno basado en **principios adaptativos (PMBOK 7)** transforma la gestión de requisitos de software, pasando de la conformidad documental a la entrega continua de valor.

1. Diferencias Fundamentales entre Enfoques

Característica	PMBOK 6ª Edición: Enfoque Centrado en Procesos	PMBOK 7ª Edición: Enfoque Basado en Principios
Filosofía Base	Prescriptiva, estructurada y predictiva.	Adaptativa, holística y orientada al valor.
Estructura	5 Grupos de Procesos, 10 Áreas de Conocimiento y 49 Procesos (ITTOs).	12 Principios de Dirección de Proyectos y 8 Dominios de Desempeño.
Gestión de Requisitos	Se considera una sub-área de la <i>Gestión del Alcance</i> . El objetivo es definir, congelar y controlar el alcance mediante una Especificación de Requisitos (SRS) estática y una Matriz de Trazabilidad rígida.	Es transversal a múltiples principios (Valor, Interesados, Adaptabilidad). Se considera una hipótesis de valor continua que evoluciona mediante el refinamiento del Backlog y el feedback empírico.
Métrica de Éxito	Conformidad con la línea base del plan inicial (Triángulo de Hierro: Alcance, Tiempo, Costo).	Entrega de <i>Outcomes</i> (resultados y beneficios de negocio) y satisfacción de necesidades reales.

Manejo del Cambio	El cambio se ve como un riesgo que debe ser controlado burocráticamente a través de un Comité de Control de Cambios (CCB).	El cambio se asume como natural y necesario para optimizar el valor; la gestión es adaptativa.
--------------------------	--	--

2. Transformación de la Gestión de Requisitos de Software

La transición al PMBOK 7 desvincula la gestión de requisitos de la rigidez de la *Gestión del Alcance* tradicional. Bajo el nuevo enfoque, el requerimiento no es un fin en sí mismo, sino un medio para aplicar los principios fundamentales de:

- **Enfoque en el Valor:** El requerimiento solo es válido si contribuye directamente a un *outcome* de negocio medible.
- **Involucramiento de Interesados:** La elicitación no es un evento único, sino una colaboración continua para asegurar la alineación.
- **Adaptabilidad y Resiliencia:** El Backlog se prioriza y refina en cada iteración, permitiendo descartar o modificar requisitos según el aprendizaje empírico.

3. Ejemplos Prácticos de la Transformación

Para ilustrar este cambio, se presentan dos escenarios comunes en el desarrollo de software:

Ejemplo 1: Descubrimiento de una nueva regla de negocio a mitad del desarrollo

- **Escenario:** Durante la construcción de un módulo de facturación, el equipo descubre que una nueva regulación fiscal afecta la lógica de cálculo.
- **Enfoque PMBOK 6 (Procesos):** El Gerente de Proyecto debe detener o retrasar la tarea, documentar formalmente una Solicitud de Cambio, someterla al CCB para análisis de impacto en costo y tiempo, y esperar aprobación formal para actualizar la SRS y la línea base del alcance antes de reanudar el desarrollo.
- **Enfoque PMBOK 7 (Principios):** Aplicando los principios de **Adaptabilidad** y **Enfoque en Valor**, el Gerente de Proyecto (o Product Owner) evalúa inmediatamente el impacto de la regulación en el beneficio del producto. Si es crítica, se refina el *Product Backlog*, se prioriza la Historia de Usuario y se integra en el siguiente Sprint, minimizando la burocracia paralizante.

Ejemplo 2: Validación de la usabilidad de una interfaz

- **Escenario:** Se ha definido un requisito funcional para una nueva pantalla de registro de usuarios.
- **Enfoque PMBOK 6 (Procesos):** El requisito se valida principalmente al final del ciclo de vida técnico (fase de pruebas de aceptación UAT), contrastando el software entregado contra la SRS firmada inicialmente. Si la interfaz es técnicamente correcta pero difícil de usar, el costo de corrección es altísimo (Curva de Boehm).
- **Enfoque PMBOK 7 (Principios):** Basado en los principios de **Involucramiento de Interesados** y **Calidad Embebida**, el Gerente de Proyecto fomenta la creación de prototipos rápidos y revisiones incrementales iterativas (*Sprint Reviews*). Los usuarios interactúan con incrementos funcionales semanalmente, permitiendo ajustar los requisitos de usabilidad sobre la marcha antes de que se genere deuda técnica.

Ejercicio 4

El Mayor Riesgo en el Software: Análisis sobre la Crisis del Software, la Ingeniería de Requisitos y la Adopción por Parte de los Actores.

1. La Crisis del Software: De los Fallos de Compilación a la Falta de Adopción en el Mundo Real

Evolución de la Crisis

- **Crisis Clásica (1968):** Enfocada en la falla técnica. Los sistemas superaban los presupuestos, no se entregaban a tiempo y colapsaron frecuentemente debido a errores de compilación y ejecución.
- **Crisis Moderna:** Enfocada en el valor de negocio. El software se construye con excelencia técnica y cero errores, pero nadie lo utiliza porque no resuelve una necesidad real.

El Factor Humano y el Riesgo de la Adopción.

Un sistema que compila perfectamente, ejecuta algoritmos complejos y no genera excepciones en el servidor carece de valor si genera una fricción inaceptable para los usuarios finales. La excelencia técnica sin adopción representa un 0 % de valor de negocio y constituye, simplemente, un desperdicio de recursos.

2. Ingeniería de Requisitos: El Puente entre Mundos

La Ingeniería de Requisitos (IR) actúa como la interfaz entre las necesidades del mundo real y la implementación técnica de software. Su propósito principal es prevenir el desarrollo de sistemas inútiles mediante un proceso continuo de licitación, análisis, especificación y validación con los interesados.

Filtro de Valor y Trampas a Evitar

- **Evitar el sesgo de solución:** No diseñar la arquitectura ni escribir código antes de comprender a fondo el problema de negocio.
- **Eliminar falsas suposiciones:** Validar directamente con los usuarios en lugar de asumir lo que necesitan para su trabajo diario.
- **Validación temprana:** Confirmar los requisitos antes de invertir semanas o meses en la fase de desarrollo.
- **Garantizar el retorno de inversión:** Asegurar que cada funcionalidad codificada responda a metas organizacionales concretas.

3. Identificación de Actores: Construir Software para Personas Reales con Roles y Flujos Concretos

El Rol de los Actores

- **No existe el "usuario promedio":** El sistema debe responder a roles, restricciones y flujos de trabajo concretos.
- **Alineación con la experiencia de usuario (UX):** Diseñar considerando el contexto operativo real evita la fricción del usuario.
- **Mitigar la resistencia al cambio:** Si el actor no percibe un beneficio directo, rechazará la herramienta.
- **Evitar sistemas huérfanos:** Se evita construir módulos complejos que ningún rol en el negocio utilizará.

Tabla 1

Análisis de Riesgos vs. Impacto en la Adopción del Software

Aspecto de Falla	Causa Principal	Consecuencia en el Negocio
------------------	-----------------	----------------------------

Mala Usabilidad (UX)	Ignorar el entorno operativo y hábitos del actor.	Uso lento y alta tasa de errores de captura.
Resistencia al Cambio	Modificar procesos sin aportar valor directo.	Rechazo total y retorno a métodos manuales.
Sistemas Huérfanos	Falta de validación de requisitos con roles clave.	Software abandonado que requirió inversión.

4. Reflexión Clave y Pilares del Éxito

"Construir perfectamente el producto equivocado es la forma más cara e inútil de fracasar en el desarrollo de software."

Pilares del Éxito (Principios de Ingeniería de Software)

Comprender: Identificar el problema real antes de proponer cualquier solución técnica o escribir una sola línea de código.

Involucrar: Mapear a los actores reales del sistema y adaptar la experiencia a sus verdaderos entornos operativos.

Validar: Verificar de forma continua los requisitos para asegurar la adopción inmediata y el impacto de negocio.

CONCLUSIONES

La conversión exacta de las necesidades empresariales en especificaciones funcionales y no funcionales estructuradas es crucial para minimizar el costo de las modificaciones. La combinación de prototipado rápido e Inteligencia Artificial Generativa mejora la validación temprana, asegurando que el sistema se ajuste a los flujos operativos reales de los usuarios antes de comenzar la construcción.

El éxito de un proyecto se basa en un balance armonioso entre las cuatro dimensiones de Pressman. Imponer arquitecturas excesivamente tecnificadas sin tener en cuenta las habilidades del equipo o faltar un proceso formal de gestión de cambios lleva invariablemente a un desbordamiento del alcance y demoras. Ante esta situación, el papel del gerente se transforma en un líder que sirve y ajusta la tecnología al entorno humano y empresarial.

La evolución del PMBOK 6 al PMBOK 7 muestra que evaluar el éxito solo por el respeto del "Triángulo de Hierro" (alcance, tiempo y costo) es inadecuado. La gestión contemporánea entiende los requisitos como hipótesis de valor en evolución que necesitan ser constantemente mejoradas a través de ciclos de retroalimentación y la participación activa de los interesados.

El principal riesgo en la ingeniería de software en la actualidad no es una falla técnica o de compilación, sino la falta de aceptación por parte de los usuarios. Un sistema libre de errores que no aporta valor ni se utiliza constituye un completo derroche de recursos; por lo tanto, el verdadero éxito de la ingeniería de software se mide únicamente en su empleo práctico en producción.